Лабораторная работа №2.
Создание проекта Web-сайта с помощью HTML

Цель работы: создать с помощью текстового редактора макет для Web-страниц сайта на языке HTML.

Общая постановки задачи

Разметить страницу так, чтобы к ней было просто применить CSS — первое, чему должен научиться будущий веб-разработчик. Такая разметка выполняется путем создания определенной структуры страниц с различным содержимым (текстовым, графическим, мультимедийным и т.д.).

В этой работе общая задача – имея контент (содержится в архиве *Content for work 2.zip*, который прилагается к данной работе) для домашней страницы веб-сайта наблюдения за птицами, добавить к нему структурные элементы, чтобы он мог использовать макет страницы.

Во ходе изучения порядка выполнения работы (см. ниже) необходимо найти решения следующим задачам работы:

1. Создать новый проект сайта с названием *BirdWatching* в соответствие со стандартной структурой файлов и каталогов сайтов.
2. Скопировать из архива все имеющеюся файлы в соответствующие директории созданного проекта.

Архив содержит:

– HTML-содержимое, к которому нужно добавить структурную разметку.

– CSS для стилизации вашей разметки на странице путем добавления элемента <link> чуть ниже существующего, указанного в начале.

– Изображения, которые используются на странице.

3. Создать макет разметки страниц будущего сайта в файле *index.html*.

Макет должен включать:

– Заголовок на всю ширину экрана, содержащий основное название страницы, логотип сайта и меню навигации. Заголовок и логотип появляются рядом друг с другом, когда применяется стилизация. Навигация располагается ниже этих двух элементов.

– Основная область содержимого с двумя столбцами: основной блок, содержащий текст приветствия, и боковая панель для размещения миниатюр изображений.

– Нижний колонтитул, содержащий информацию об авторских правах и разработчиках.

4. Применить подходящую обертку для:

– Заголовка.
– Меню навигации.
– Основного содержимого.
– Приветственного текста.
– Боковой панели изображения.
– Нижнего колонтитула.

5. Подключить CSS-файл со стилем из архива к макету в файле *index.html*.

Следующий скриншот показывает пример того, как может выглядеть домашняя страница после применения требований.



Порядок выполнения работы

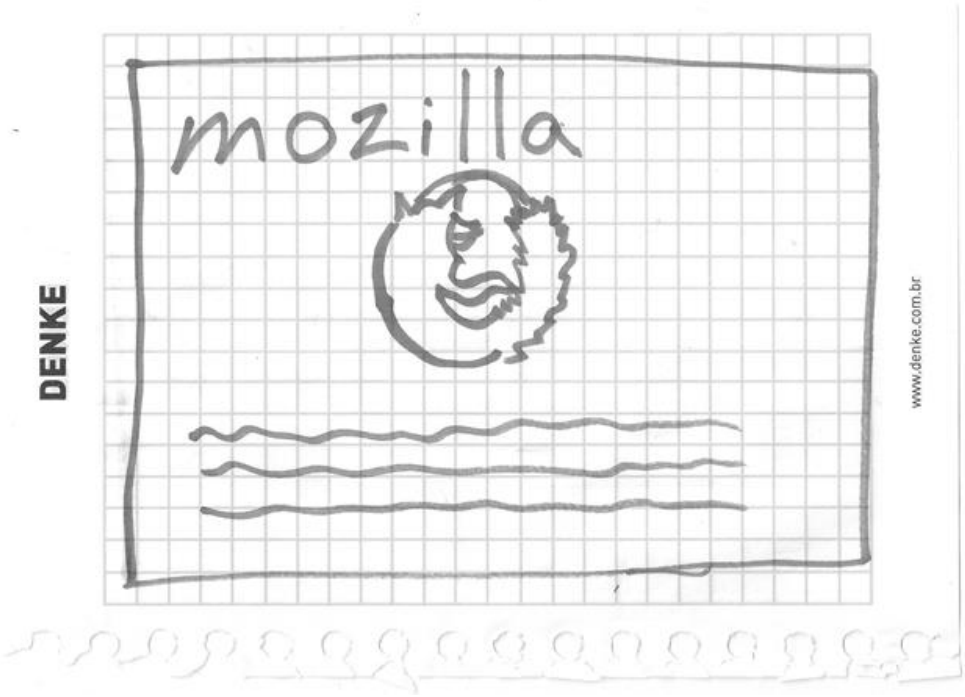
Перед тем как создавать собственный проект, нужно сформулировать основную идею: что веб-сайт должен фактически делать? В принципе веб-сайт может делать многие вещи, но для начала, лучше выбрать что-то простое. Начнем с создания простой веб-страницы, содержащей заголовок, изображение и несколько абзацев.

Для начала, вам будет нужно ответить на следующие вопросы:

1. **О чем ваш веб-сайт?** Вам нравятся собаки, Нью-Йорк или Pacman?
2. **Какую информацию вы предоставляете о предмете?** Напишите заголовок и несколько абзацев, и подумайте над изображениями, которые вы хотите показать на своей странице.
3. **Как будет выглядеть ваш веб-сайт**, в простых терминах высокого уровня. Какой цвет фона? Какой вид шрифта будет уместен: деловой, мультяшный, жирный и кричащий или тонкий?

Комплексные проекты нуждаются в детализированных руководствах, которые включают все детали цветов, шрифтов, расстояния между элементами на странице, соответствующий стиль письма и так далее. Их иногда называют руководствами по проектированию или бренд-бук.

Теперь возьмите ручку и бумагу и сделайте примерный набросок того, как вы хотите, чтобы выглядел ваш сайт. Для вашей первой веб-страницы должен получиться небольшой набросок, и вы должны взять это в привычку. Это действительно помогает, и вам не нужно быть Ван Гогом!



Даже в реальных, сложных веб-сайтах, команда разработчиков обычно начинает с наброска на бумаге и потом строит цифровые макеты используя графические редакторы или веб-технологии.

Веб-команда часто включает в себя пару графических дизайнеров и дизайнера с опытом взаимодействия (user-experience (UX) designer). Графические дизайнеры, очевидно, работают вместе над визуализацией веб-сайта. UX дизайнеры играют более абстрактную роль, обращаясь к тому как пользователи будут пользоваться и взаимодействовать с веб-сайтом.

Веб-страницы могут и будут отличаться друг от друга, но все они, преимущественно, состоят из аналогичных стандартных компонентов, если только страница не отображает полноэкранное видео или игру, не является частью какого-либо художественного проекта или просто плохо структурирована:

Заголовок - обычно это большая полоса вверху страницы, с крупным заголовком и / или логотипом. Здесь указывается общая информация о веб-сайте, не меняющаяся от страницы к странице.

Навигационное меню - ссылки на основные разделы сайта; обычно в виде кнопок, ссылок или вкладок. Также как и заголовок, навигация остается неизменной на всех страницах сайта – наличие непоследовательной навигации на Вашем сайте запутает и разочарует пользователей. Многие веб-дизайнеры считают панель навигации частью заголовка, а не отдельным компонентом, но это не является обязательным требованием; на самом деле, некоторые также утверждают, что их

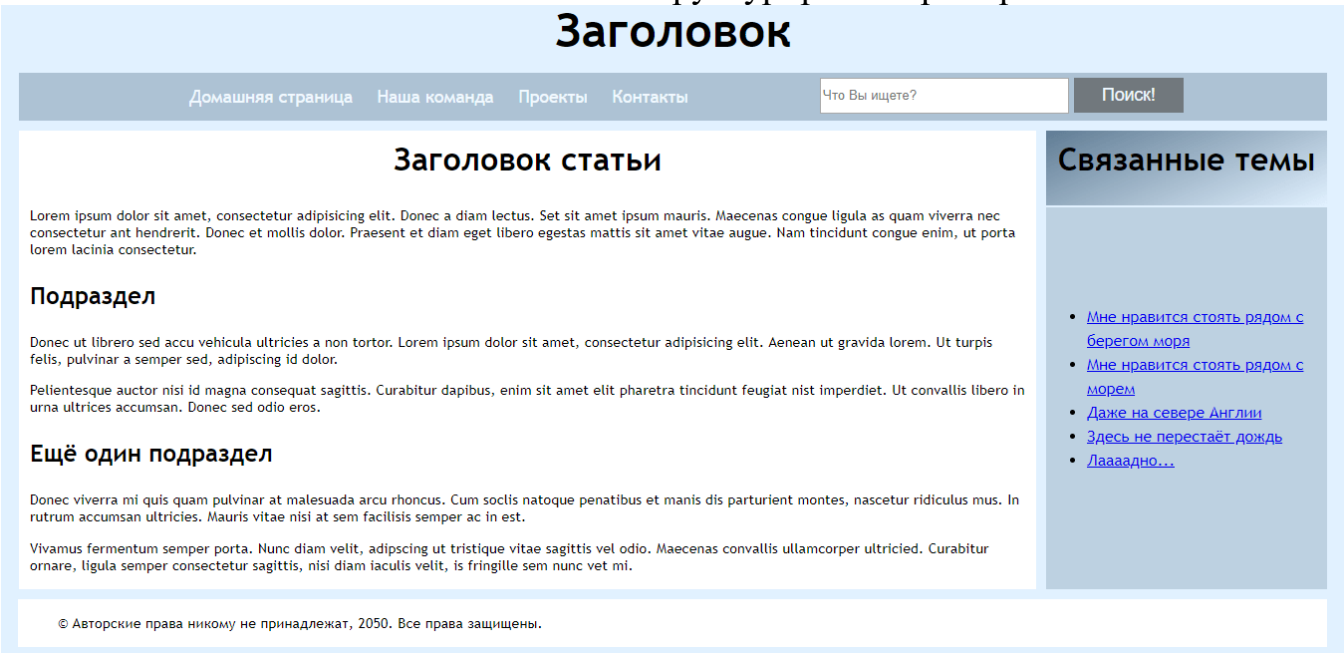
разделение на отдельные компоненты улучшает доступность, поскольку раздельная структура будет понятнее для людей, пользующихся считывателями экрана.

Основное содержимое – большая область в центре страницы, содержащая, в основном, уникальный контент данной веб-страницы, например видео, которое вы хотите посмотреть, или рассказ, который вы читаете, или карту, которую вы хотите просмотреть, или заголовки новостей и т. д. Это одна из частей сайта, которая определенно будет меняться от страницы к странице!

Боковая панель - как правило, содержит некоторую второстепенную информацию, ссылки, цитаты, рекламу и т.д. Обычно она относится к содержимому в основном контенте (например, на странице со статьей, боковая панель может содержать биографию автора или ссылки на связанные статьи), но в некоторых случаях здесь размещают и другие элементы, например, вторичную навигационную систему.

Нижний колонтитул (футер) – полоса в нижней части страницы, которая обычно содержит уведомления об авторских правах или контактную информацию. Это место для размещения общей информации (например, заголовка), но обычно эта информация не является критичной или вторична для самого веб-сайта. Нижний колонтитул также иногда используется для SEO целей, предоставляя ссылки для быстрого доступа к популярному контенту.

"Типичный веб-сайт" может быть структурирован примерно так:



Пример, показанный сверху, не красив и примитивен, но идеально подходит для иллюстрирования типичного макета веб-сайта. У некоторых веб-сайтов больше колонок, некоторые – более сложные, но общая идея очевидна.

Визуальные эффекты – это ещё не самое главное. Мы используем цвет и размер шрифта для привлечения внимания посетителей к наиболее полезным частям содержимого, такого как навигационное меню или связанные ссылки, но что насчет людей со слабым зрением, к примеру, для которых концепция "розового" и "большого шрифта" не будет полезной?

В своём HTML-коде Вы можете размечать разделы содержимого сайта на основе их *функциональности* – использовать элементы, которые представляют разделы контента, описанные выше, а вспомогательные технологии, такие как

программы чтения с экрана, смогут распознавать эти элементы и помогать в таких задачах, как "найти основную навигацию" или "найти основное содержимое". Как мы упоминали ранее в ходе курса, существует ряд последствий неиспользования правильной структуры элементов и семантики для правильной работы.

В спецификации HTML до версии 4.01 выделялось две основные категории HTML-элементов, которые соответствуют типам их содержимого и поведению в структуре веб-страницы – блочные и строчные элементы.

С помощью блочных элементов можно создавать структуру веб-страницы. Элементы блочного уровня формируют видимый блок на странице – они окажутся на новой строке после любого контента, который шёл до них, и любой контент после них также окажется на новой строке. Чаще всего элементами блочного уровня бывают структурные элементы страницы, представляющие собой, например, параграфы (абзацы), списки, меню навигации, футеры, или подвалы, и т. п. Элементы блочного уровня не вкладываются в строчные элементы, но иногда могут вкладываться в другие элементы блочного уровня.

Строчные элементы – это те, которые содержатся в элементах блочного уровня и окружают только малые части содержимого документа, не целые абзацы и группировки контента. Строчные элементы не приводят к появлению новой строки в документе: они обычно встречаются внутри абзаца текста, например, элемент <a> (ссылка) или акцентирующие элементы вроде или . Строчные элементы используются для форматирования текстовых фрагментов (за исключением элементов <area> и <img>).

В HTML5 эти понятия заменены более сложным набором категорий контента, согласно которым каждый HTML-элемент должен следовать правилам, определяющим, какой контент для него допустим.

Согласно спецификации HTML5 каждый элемент принадлежит к определенной (ноль или более) категории. Каждая из них группирует элементы со схожими характеристиками. Выделяют следующие общие категории:

- Метаданные
- Потоковое содержимое
- Секционное содержимое
- Заголовочное содержимое
- Текстовое содержимое
- Встроенное содержимое
- Интерактивное содержимое

Секционное, заголовочное, текстовое, встроенное и интерактивное содержимое относятся к потоковому содержимому. Мета содержимое иногда может относиться к потоковому содержимому. Мета содержимое и интерактивное содержимое иногда относятся к текстовому содержимому. Встроенное содержимое также относится к текстовому содержимому, но часть элементов также является интерактивным содержимым.

Другие категории используются для специфических целей, например, для определения элементов форм. Некоторые элементы имеют уникальные требования и не вписываются в какую-либо конкретную категорию.

Мета содержимое

<base>, <link>, <meta>, <noscript>, <script>, <style>, <template>, <title>

К мета содержимому относятся элементы, которые содержат информацию о HTML-документе, устанавливают взаимосвязь HTML-документа с другими документами (например, таблицами стилей css), отвечают за внешний вид или поведение контента на странице. Элементы из другого пространства имён, чей смысл связан с метаданными, также относятся к мета содержимому.

Потоковое содержимое

<a>, <abbr>, <address>, <area> (если он является потомком элемента <map>), <article>, <aside>, <audio>, <b>, <bdi>, <bdo>, <blockquote>,
, <button>, <canvas>, <cite>, <code>, <data>, <datalist>, <del>, <details>, <dfn>, <dialog>, , <dl>, , <embed>, <fieldset>, <figure>, <footer>, <form>, <h1>, <h2>, <h3>, <h4>, <h5>, <h6>, <header>, <hr>, <i>, <iframe>, <img>, <input>, <ins>, <kbd>, <label>, <link> (если его присутствие внутри тела документа допускается), <main>, <map>, <mark>, MathML, <math>, <menu>, <meter>, <nav>, <noscript>, <object>, <ol>, <output>, <p>, <picture>, <pre>, <progress>, <q>, <ruby>, <s>, <samp>, <script>, <section>, <select>, <small>, , , <sub>, <sup>, SVG, <svg>, <table>, <template>, <textarea>, <time>, <u>, <ul>, <var>, <video>, <wbr>, текст

К этому типу содержимого относится большинство элементов, используемых в теле документа или веб-приложения.

Секционное содержимое

<article>, <aside>, <nav>, <section>

Секционное содержимое – содержимое, определяемое областью заголовков и колонтитулов. Каждый элемент секционного содержимого потенциально имеет заголовок и схему (outline).

Заголовочное содержимое

<h1>, <h2>, <h3>, <h4>, <h5>, <h6>

Заголовки разделов, размеченных явно с помощью элементов секционного содержимого, либо просто подразумеваемых наличием заголовка.

Текстовое содержимое

<a>, <abbr>, <area> (если он является потомком элемента <map>), <audio>, <b>, <bdi>, <bdo>,
, <button>, <canvas>, <cite>, <code>, <data>, <datalist>, <del>, <dfn>, , <embed>, <i>, <iframe>, <img>, <input>, <ins>, <kbd>, <label>, <link> (если его присутствие внутри тела документа допускается), <map>, <mark>, MathML, <math>, <meter>, <noscript>, <object>, <output>, <picture>, <progress>, <q>, <ruby>, <s>, <samp>, <script>, <select>, <small>, , , <sub>, <sup>, SVG, <svg>, <template>, <textarea>, <time>, <u>, <var>, <video>, <wbr>, текст

Текстовое содержимое представлено элементами для разметки текста внутри абзацев. Большинство элементов, принадлежащих категории текстовое содержимое, могут содержать только элементы из своей категории, а не любые элементы из категории потокового содержимого.

Встроенное содержимое

<audio>, <canvas>, <embed>, <iframe>, <img>, MathML, <math>, <object>, <picture>, SVG, <svg>, <video>

Встроенное содержимое – содержимое, которое импортируется в документ из других источников, другого языка разметки или пространства имён. Некоторые элементы могут иметь резервный контент, который будет задействован, если внешний ресурс не может быть использован (например, в случае неподдерживаемого формата видео).

Интерактивное содержимое

<a> (если присутствует атрибут href), <audio> (если присутствует атрибут controls), <button>, <details>, <embed>, <iframe>, <img> (если присутствует атрибут usemap), <input> (если значение атрибута type не равно hidden), <label>, <select>, <textarea>, <video> (если присутствует атрибут controls)

Интерактивное содержимое предполагает взаимодействие с пользователем. Атрибут tabindex также может сделать любой элемент интерактивным содержимым.

Явное содержимое

<a>, <abbr>, <address>, <article>, <aside>, <audio> (если присутствует атрибут controls), <b>, <bdi>, <bdo>, <blockquote>, <button>, <canvas>, <cite>, <code>, <data>, <details>, <dfn>, , <dl> (если он содержит по крайней мере одну пару термин-описание), , <embed>, <fieldset>, <figure>, <footer>, <form>, <h1>, <h2>, <h3>, <h4>, <h5>, <h6>, <header>, <i>, <iframe>, <img>, <input> (если значение атрибута type не равно hidden), <ins>, <kbd>, <label>, <main>, <map>, <mark>, MathML, <math>, <menu> (если значение атрибута type равно toolbar), <meter>, <nav>, <object>, <ol> (если он содержит по крайней мере один элемент <li>), <output>, <p>, <pre>, <progress>, <q>, <ruby>, <s>, <samp>, <section>, <select>, <small>, , , <sub>, <sup>, SVG, <svg>, <table>, <textarea>, <time>, <u>, <ul> (если он содержит по крайней мере один элемент <li>), <var>, <video>, текст, не являющийся пробелом между элементами

Как правило, элементы, принадлежащие модели потокового или текстового содержимого, должны иметь минимум один узел, который имеет явное содержимое и для него не задан атрибут hidden. Явное содержимое делает элемент непустым. Это обеспечивается тем, что элемент содержит потомка с текстом или каким-либо содержимым, которое пользователь может слышать (элемент <audio>), видеть

(элемент <canvas>, <img> или <video>) или каким-либо образом взаимодействовать с ним (интерактивные элементы форм).

Это требование не является жёстким, так как в некоторых случаях элемент может быть пустым по очевидным причинам (например, он в последствии будет наполняться с помощью скрипта).

Элементы, поддерживающие скрипт

<script>, <template>

Элементы, поддерживающие скрипт, не отображаются на экране. Они используются для поддержки сценариев, например, для обеспечения функциональных возможностей для пользователя.

Корневое секционное содержимое

<blockquote>, <body>, <details>, <dialog>, <fieldset>, <figure>, <td>

Кроме секционного содержимого есть несколько элементов, являющихся корневыми секционными. Они выделены из секционного содержимого, но также могут иметь собственную структуру заголовков, а разделы и заголовки внутри этих элементов не взаимодействуют со структурой заголовков их родителей.

Прозрачная модель содержимого

<a>, <audio>, <canvas>, <del>, <ins>, <map>, <object>, <video>

Некоторые элементы имеют прозрачную модель содержимого. Это означает, что тип ожидаемого содержимого наследуется от их родительского элемента. Дополнительно к собственному разрешенному контенту они могут содержать любой контент, который допустим для их родительского элемента. Если такой элемент не имеет родительского элемента, то его прозрачная модель контента должна рассматриваться как ожидающая любое содержимое из категории потокового содержимого.

Для реализации семантической разметки на HTML наиболее часто используются следующие выделенные теги:

- **Заголовок:** <header>.
- **Навигационное меню:** <nav>.
- **Основное содержимое:** <main>, с различными подразделами содержимого, представленными элементами <article>, <section> и .
- **Боковая панель:** <aside>, обычно располагается внутри <main>.
- **Нижний колонтитул:** <footer>.

Полезно понять общий смысл всех структурных элементов HTML – это то, над чем вы будете работать постепенно, когда начнёте получать больше опыта с веб-разработкой.

- <main> предназначен для содержимого, *уникального для этой страницы*. Используйте <main> только *один* раз на странице и размещайте прямо внутри <body>. В идеале он не должен быть вложен в другие элементы.
- <article> окружает блок связанного содержимого, который имеет смысл сам по себе без остальной части страницы (например, один пост в блоге).
- <section> подобен <article>, но больше подходит для группирования одной части страницы, которая представляет собой одну часть функциональности (например, мини-карту или набор заголовков статей и сводок). Считается хорошей практикой начинать каждый раздел с заголовка. Также обратите внимание, что в

зависимости от контекста вы можете разбить <article> на несколько <section> или, наоборот, <section> на несколько <article>.

– <aside> содержит контент, который не имеет прямого отношения к основному содержимому, но может содержать дополнительную информацию, косвенно связанную с ним (словарь, биография автора, связанные ссылки и т.д.).

– <header> представляет собой группу вводного содержимого. Если он дочерний элемент <body>, то он определяет глобальный заголовок веб-страницы, но если он дочерний элемент <article> или <section>, то определяет конкретный заголовок для этого раздела (постарайтесь не путать его с titles и headings).

– <nav> содержит основные функции навигации для страницы. Так же часто в нем можно увидеть логотип и / или название сайта или компании. Вторичные ссылки и т.д. не входят в навигацию.

– <footer> представляет собой группу конечного контента для страницы.

Иногда Вы будете сталкиваться с ситуацией, когда Вы не можете найти идеальный семантический элемент, чтобы сгруппировать некоторые элементы вместе или обернуть некоторый контент. Иногда Вам просто нужно будет сгруппировать несколько элементов вместе, чтобы применить к ним, как к единой сущности, CSS или JavaScript. Для таких случаев в HTML есть элементы и . Вам следует использовать их с подходящим значением атрибута class или id, чтобы можно было легко получить к ним доступ.

 – это строчный несемантический элемент, который стоит использовать только если Вы не можете подобрать более подходящий семантический текстовый элемент для обёртывания контента или если не хотите добавлять какие-либо конкретные значения. Например:

```
<p>Пьяный Король возвратился в свою комнату в 01:00
и всё никак не мог войти в дверь: хмель мешал <span class="editor-note">[Примечание редактора: В этот момент
свет на сцене должен быть приглушён]</span>.</p>
```

В этом примере примечание редактора просто сообщает дополнительные пожелания режиссёру пьесы. В нем нет особого семантического значения. Для слабовидящих пользователей, возможно, примечание будет отделено от основного содержимого с помощью CSS.

 – это блочный несемантический элемент, который следует использовать только если Вы не можете подобрать более подходящий семантический блочный элемент или если не хотите добавлять какие-либо конкретные значения. Например, представьте виджет корзины в интернет-магазине, который Вы можете открыть в любой момент нахождения на сайте:

```
<div class="shopping-cart">
  <h2>Корзина</h2>
  <ul>
    <li>
      <p><a href=""><strong>Silver earrings</strong></a>: $99.95.</p>
      <img src="../products/3333-0985/thumb.png" alt="Серебряные серьги">
    </li>
```

```
    <li>
      ...
    </li>
  </ul>
  <p>Итого: $237.89</p>
</div>
```

Ему не подходит <aside>, поскольку это не обязательно относится к основному содержимому страницы (Вы хотите, чтобы его можно было просматривать из любого места). Также не подходит и <section>, т. к. это не часть основного содержимого страницы. Поэтому подходит в этом случае. Мы включили заголовок в качестве указателя, чтобы помочь пользователям программ чтения с экрана в его поиске.

Div настолько просто использовать, что легко переборщить. Поскольку они не несут никакого семантического значения, они просто загромождают Ваш HTML-код. Старайтесь использовать их только тогда, когда нет лучшего семантического решения, и постарайтесь свести их использование к минимуму, иначе Вам будет трудно обновлять и поддерживать Ваши документы.

Существует еще два часто используемых элемента:
 и <hr>:

 создает разрыв строки в абзаце, и это единственный способ изменить жёсткую структуру в ситуации, когда Вам нужна серия фиксированных коротких строк, например, в почтовом адресе или стихотворении. Пример:

```
<p>Жила-была девчушка Нелл,<br>
Любившая писать HTML:<br>
Её семантика ужасна была − <br>
Она и сама прочитать ничего не могла.</p>
```

Без элемента
 абзац разместится в одну длинную линию, а с ним в коде – разметка будет выглядеть следующим образом:

Жила-была девчушка Нелл,
Любившая писать HTML:
Её семантика ужасна была –
Она и сама прочитать ничего не могла.

<hr> создает горизонтальный разделитель в документе, это означает тематическое изменение текста (например, изменение темы или сцены). Визуально он просто похож на горизонтальную линию. В качестве примера:

```
<p>Рон был зажат в углу адскими тварями. Он боялся, но твёрдо решил защитить своих друзей, поднял свою волшебную палочку и приготовился к битве, надеясь, что справится со своим несчастьем.</p>
<hr>
<p>Тем временем Гарри сидел дома с раскрытым указом и размышлял о том, когда выйдут новые серии спин-оффов; в это время зачарованное письмо порхнуло в окно и приземлилось у него на коленях. Он прочитал его и подскочил на ноги; он подумал: "Думаю, самое время вернуться к работе".</p>
```

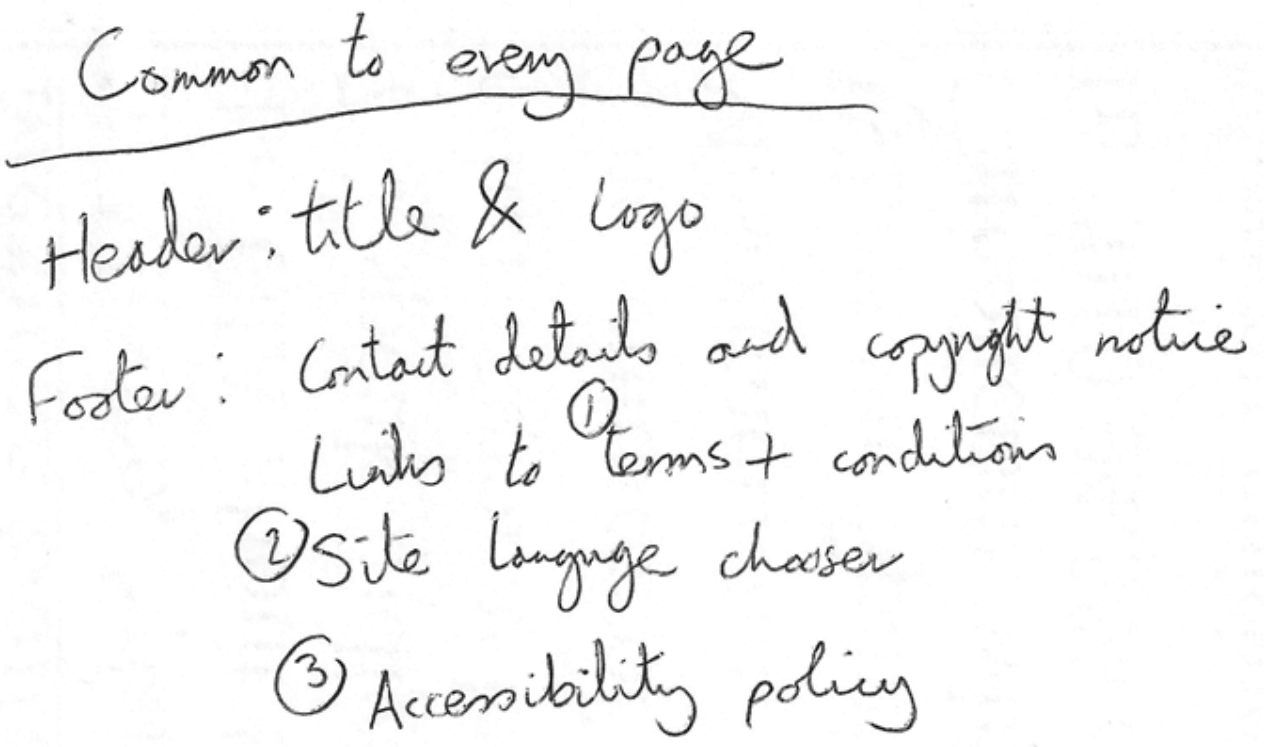
Будет выглядеть примерно так:

Рон был зажат в углу адскими тварями. Он боялся, но твёрдо решил защитить своих друзей, поднял свою волшебную палочку и приготовился к битве, надеясь, что справится со своим несчастьем.

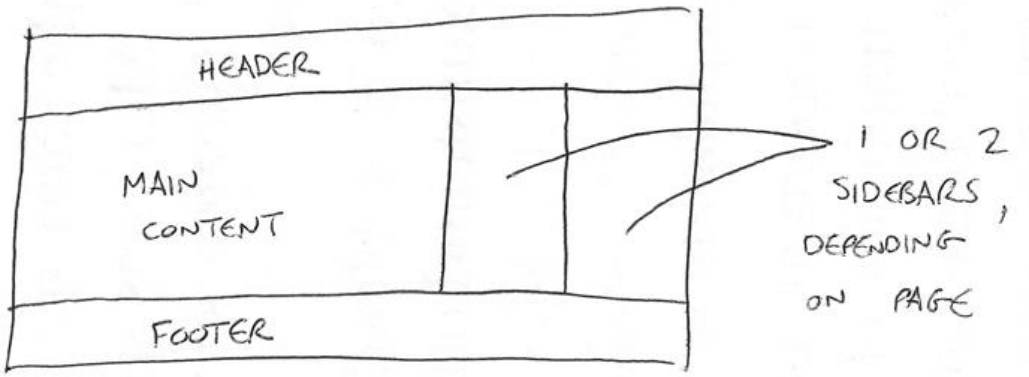
Тем временем Гарри сидел дома с раскрытым указом и размышлял о том, когда выйдут новые серии спин-оффов; в это время зачарованное письмо порхнуло в окно и приземлилось у него на коленях. Он прочитал его и подскочил на ноги; он подумал: "Думаю, самое время вернуться к работе".

Когда вы уже спланировали содержание одной веб-страницы, следующий логический шаг – продумать содержание всего веб-сайта: какие страницы нужны, как они будут устроены и связаны друг с другом для лучшего восприятия пользователем. Это называется Information architecture. В большом, сложном веб-сайте на планирование может уходить много времени, однако спроектировать простой веб-сайт из нескольких страниц может быть очень легко и весело!

1. Имейте в виду, что у вас будет несколько элементов, общих для большинства (если не всех) страниц – например, меню навигации и содержимого нижнего колонтитула. Например, для сайта компании хорошая идея разместить контактные данные в нижнем колонтитуле на каждой странице. Составьте список элементов, общих для всех страниц.



2. Теперь набросайте структуру страниц (можно взять за образец наш простой дизайн, приведенный раннее). Что находится в этих блоках?



3. Теперь составьте список остальной (уникальной для каждой страницы) информации, которую вы разместите на сайте.

Search for Flights
Hotels / other accommodation
Transport
Things to do

Special offers

Popular holiday packages
e.g. Winter sun
Disneyworld
Skiing

Search results

Country-specific info

Accommodation / attraction reviews

Visa / entry requirements

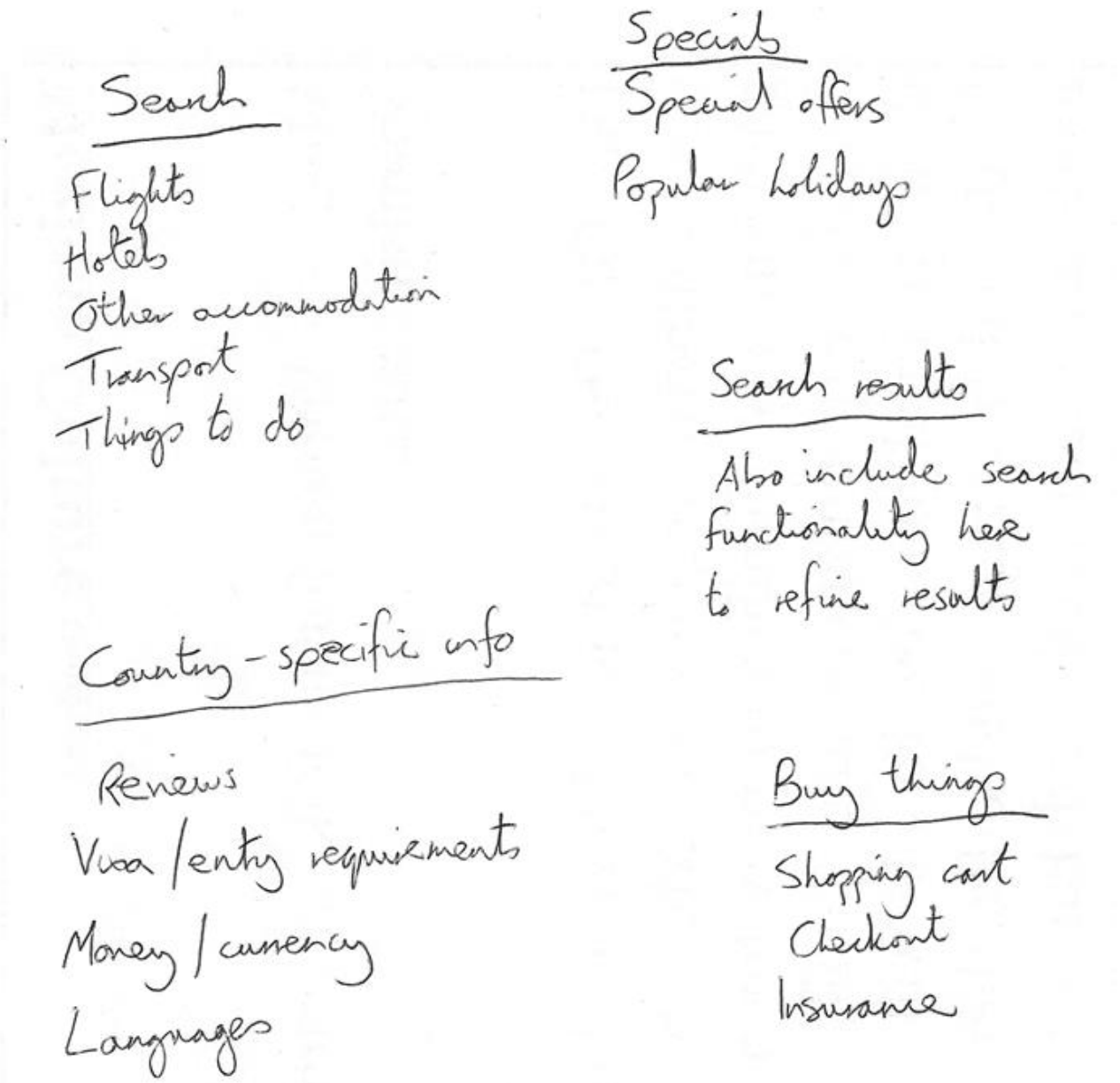
Money / Currency

Languages

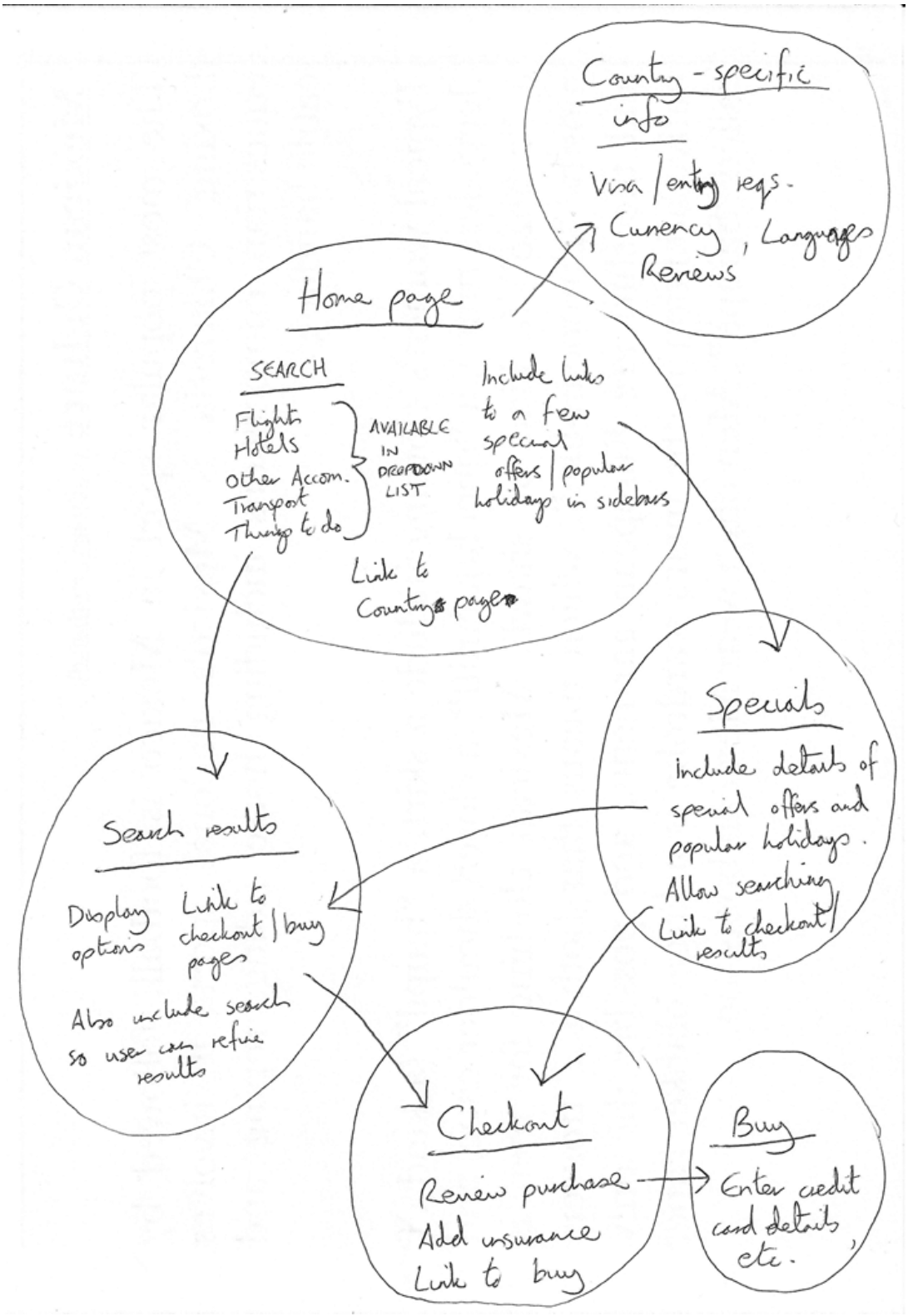
Insurance

Buy holidays / other things

4. Сгруппируйте информацию по темам. Какие части можно разместить на одной странице? Это похоже на метод Card sorting.



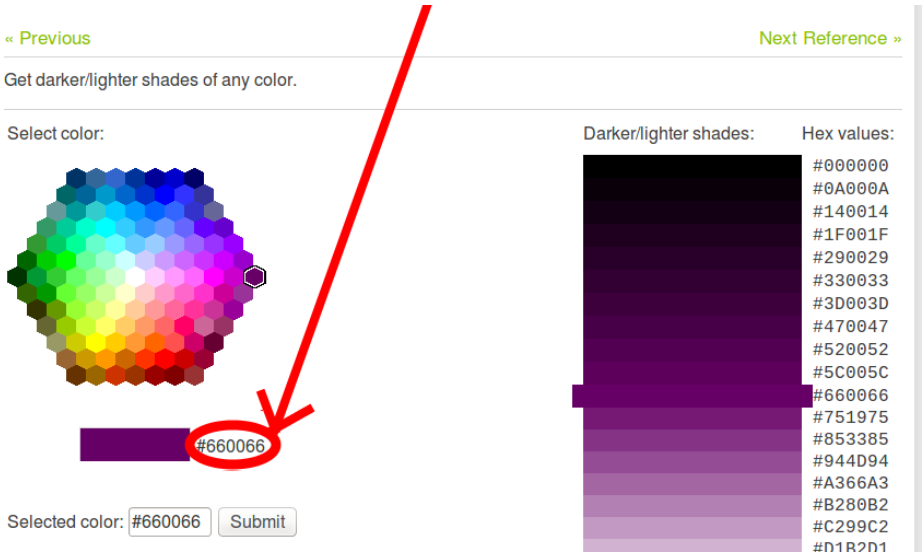
5. Составьте карту сайта. Обведите каждую страницу рамкой, и продумайте перемещения пользователя между ними. Обычно в центре оказывается главная страница, с которой можно быстро перейти на все остальные. На небольшом сайте большинство страниц помещают в главную навигацию, но не обязательно класть туда все ссылки. Также можете пометить, как выглядят элементы страниц – ссылками, списками, карточками.



На начальном этапе реализации проекта необходимо начать собирать контент, который в конечном итоге появится на вашей веб-странице.

У вас должен быть **текст**, разбитый на заголовки и параграфы.

Чтобы выбрать **цвета** для контента, которые вам нравится, используйте онлайн-инструменты выбора цвета. Когда вы щёлкните по цвету, вы увидите код из шести цифр, например, #660066 (*шестнадцатеричный код (hex(adecimal) code)*. Он соответствует некоторому цвету.



Скопируйте этот код в свой проект (на бумагу или в новый файл).

Чтобы выбрать **изображение**, перейдите, например, в Google Картинки и найдите что-нибудь подходящее для своей идеи:

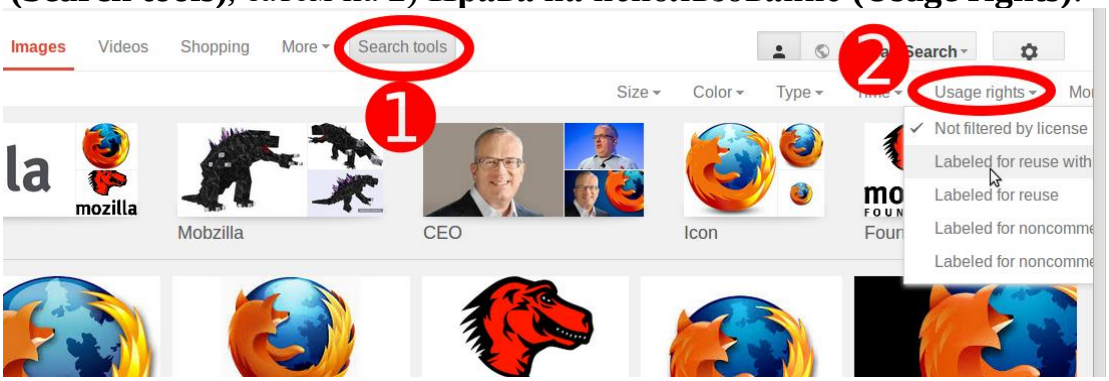
1. Когда найдете нужное изображение, щёлкните по нему.
2. Нажмите кнопку *В полном размере (View image)*.



3. На следующей странице, правым щелчком мыши на изображении (Ctrl + клик на Mac), выберите Сохранить изображение как... (*Save Image As...)*, и выберите место для сохранения изображения. В качестве альтернативы, скопируйте адрес изображения из адресной строки браузера для последующего использования.

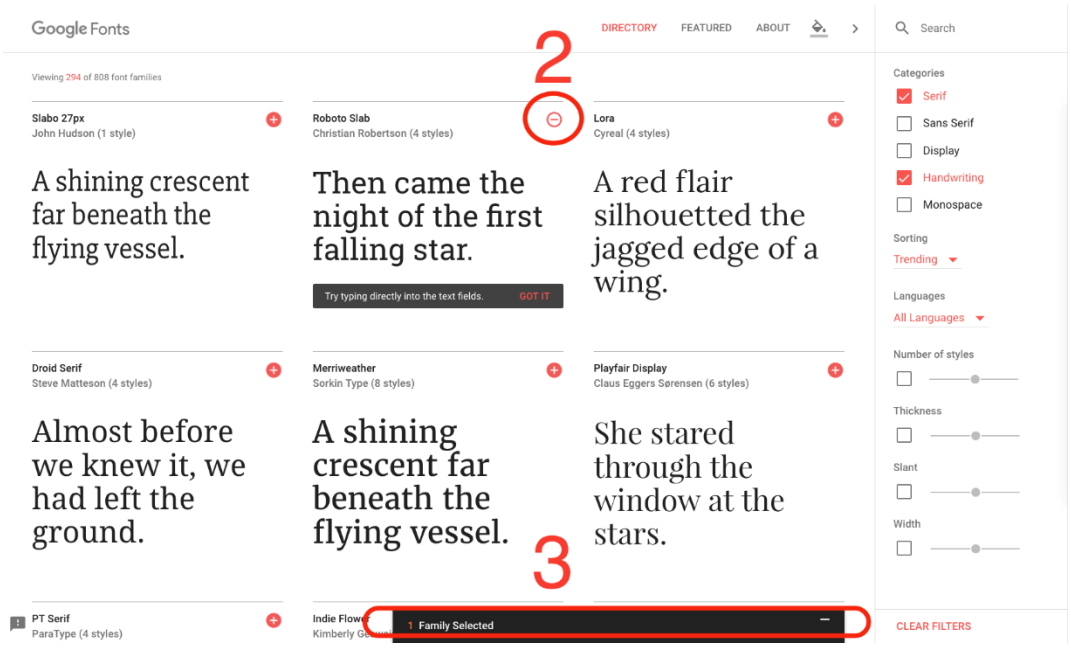


Большинство изображений в Интернете, использованных в Google Картинках имеют авторские права. Для снижения вероятности нарушения авторских прав, используйте фильтр лицензии Google. Для этого: 1) кликните на **Инструменты поиска (Search tools)**, затем на 2) **Права на использование (Usage rights)**:

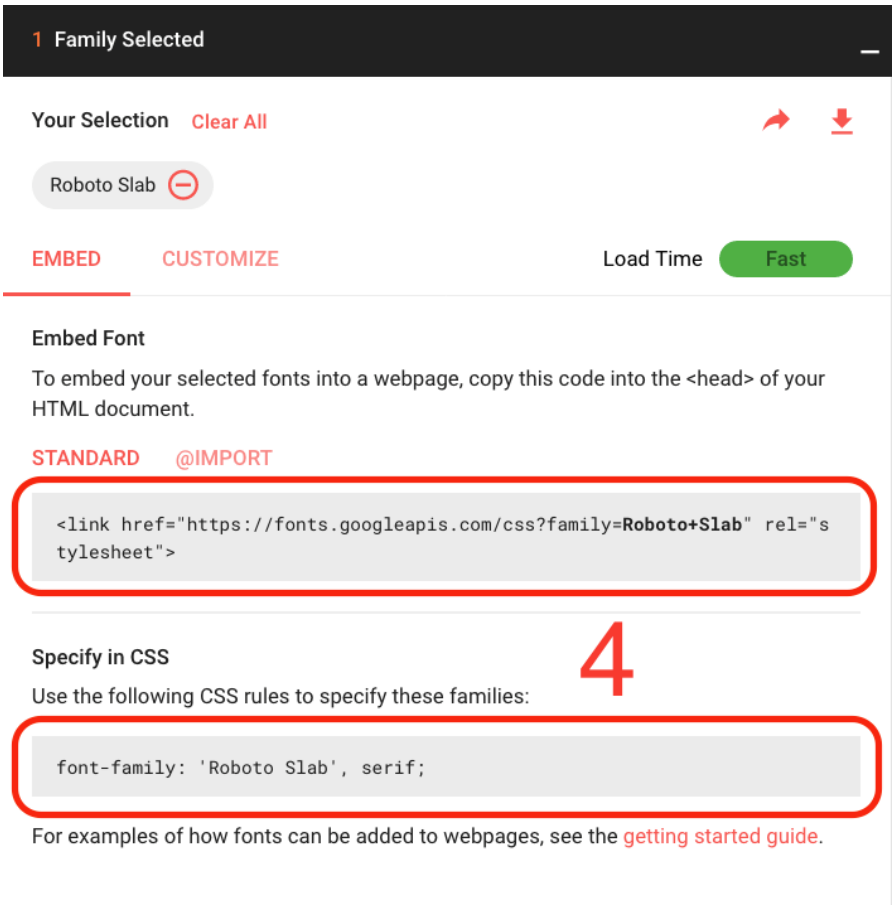


Чтобы выбрать шрифт:

1. Перейдите на Google Fonts и прокрутите список вниз, пока не найдете шрифт, который вам понравится. Вы также можете использовать элементы управления справа для дальнейшей фильтрации результатов.
2. Щёлкните по кнопке «плюс» рядом со шрифтом, который вы хотите выбрать.
3. Щёлкните по кнопке «*Family Selected» на панели в нижней части страницы («*» зависит от того, сколько шрифтов вы выбрали).



4. Во всплывающем окне вы можете увидеть и скопировать строки кода, которые предоставляет Google, чтобы сохранить их позже в вашем текстовом редакторе.



В начале Web был просто текстом, что было довольно скучно. К счастью, это продолжалось не долго – до появления возможности вставлять изображения (и другие, более интересные, типы контента) в веб-страницы. Существуют и другие типы мультимедиа, однако логичнее начать со скромного <img> элемента, используемого для вставки простого изображения в веб-страницу.

Чтобы продемонстрировать это, вставим немного HTML в наш файл index.html и научим его отображать некоторое изображение.

1. Скопируйте одно из изображений, которое вы выбрали ранее, в папку images.

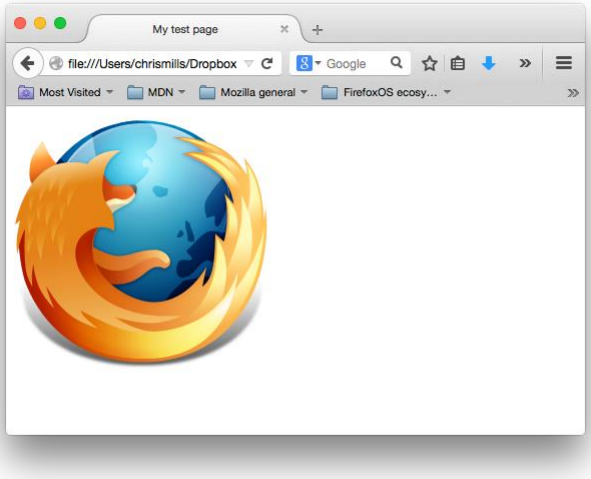
2. Откройте файл index.html и вставьте следующий код в файл в следующем виде:

```
<!DOCTYPE html>
<html>
 <head>
  <meta charset="utf-8">
  <title>Моя тестовая страница</title>
 </head>
 <body>
  <img src="" alt="Моё тестовое изображение">
 </body>
</html>
```

3. Строка <img src="" alt="Моё тестовое изображение"> – это HTML-код, который вставляет изображение на страницу. Необходимо указать интерпретатору HTML, где находится изображение. В нашем случае изображение находится внутри папки *images*, которая находится в той же директории что и index.html. Чтобы спуститься вниз по нашей файловой структуре от index.html до нашего изображения, путь к файлу должен выглядеть так – images/*your-image-filename*. Например для изображения с именем firefox-icon.png имеем такой путь к файлу: images/firefox-icon.png.

4. Вставьте путь к файлу в ваш HTML код между двойными кавычками src="".

5. Сохраните HTML файл, а затем загрузите его в браузере (двойной щелчок по файлу). Вы должны увидеть новую веб-страницу, отображающую изображение.



Код встраивает изображение на страницу в нужном месте. Это делается с помощью атрибута src (source, источник), который содержит путь к нашему файлу изображения. Атрибут src содержит путь к изображению которое вы хотите встроить в страницу, который может быть относительным или абсолютным URL.

Поисковые системы также читают имена изображений и считают их для оптимизации поискового запроса. Поэтому, давайте вашим изображениям смысловые имена; dinosaur.jpg лучше, чем img835.png.

Большинство изображений защищены. Не отображайте изображения на вашем сайте пока:

1) вы не будете владеть изображением;

2) у вас не будет письменного разрешения владельца изображения;

3) пока у вас не будет достаточно доказательств что изображение находится в открытом доступе.

Нарушение авторских прав является не законным. Кроме того, никогда не указывайте в своем атрибуте src ссылку на изображение, размещенную на чужом сайте. Это называется "хотлинкинг" (с англ. 'hotlinking' - 'горячая ссылка'). Запомните, кража чей-то пропускной способности не законна. Кроме того, это замедляет вашу страницу, оставляя вас без контроля над изображением; было ли оно удалено или случайно перемещено.

Также используется атрибут alt (alternative, альтернатива). В этом атрибуте указывается поясняющий текст для пользователей, которые не могут увидеть изображение, возможно, по следующим причинам:

1. У них присутствуют нарушения зрения. Пользователи со значительным нарушением зрения часто используют инструменты, называемые Screen Readers (экранные дикторы), которые читают для них альтернативный текст.

2. Что-то пошло не так, в результате чего изображение не отобразилось. Например, попробуйте намеренно изменить путь в вашем атрибуте src, сделав его неверным. Если вы сохраните и перезагрузите страницу, то вы должны увидеть вместо изображения надпись: My test image .

Альтернативный текст – это «пояснительный текст». Он должен предоставить читателю достаточно информации, чтобы иметь представление о том, что передает изображение. В этом примере текст "My test image" ("Моё тестовое изображение") не годится. Намного лучшей альтернативой для нашего логотипа Firefox будет "The Firefox logo: a flaming fox surrounding the Earth" ("Логотип Firefox: огненный Лис вокруг Земли").

Что именно вы должны писать в атрибут alt? В первую очередь, это зависит от того, *зачем* изображение вообще находится на странице. Другими словами, что вы потеряете, если ваше изображение не появится:

- **Декорация.** Если изображение служит просто украшением и не является частью содержимого, добавьте пустой alt="". Например, программа чтения с экрана не тратит время на чтение содержимого, которое не является важным для пользователя. На самом деле, декоративные изображения не принадлежат вашему HTML. CSS background images должны быть использованы для вставки декораций, но, если это неизбежно, то лучшее решение – использовать alt="".
- **Контент.** Если ваше изображение содержит важную информацию, передайте ту же информацию через краткий alt. Или даже лучше, в *главном* тексте, который все увидят. Не используйте alt, если можете обойтись без него. Насколько неудобно было бы для пользователя если бы *параграфы* были написаны дважды в

главном контенте? Если изображение адекватно описано в основном тексте, можете просто использовать alt="".

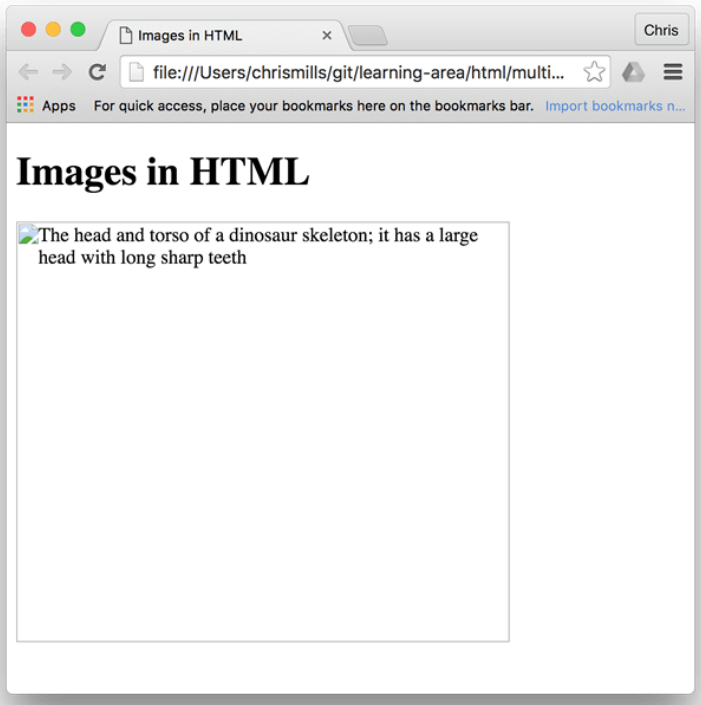
- **Ссылка.** Если вы помещаете изображение в <a>, для того, чтобы сделать из него ссылку, вы всё ещё должны предоставить accessible link text. В таком случае, вы сможете использовать элемент <a> или атрибут alt. Старайтесь выбрать лучший вариант.
- **Текст.** Не пишите текст в изображениях. Если вашему заголовку понадобиться тень, то лучше используйте для этого CSS вместо добавления текста в изображение. Однако, если *действительно этого не избежать*, то вам следует дополнить текст в атрибуте alt.

По существу, главная идея здесь это предоставить нечто полезное, для случая когда изображения не видны. Это гарантирует что все пользователи не упустят ничего из содержимого страницы. Попробуйте отключить изображения в своём браузере и посмотрите как всё выглядит. Вы вскоре выясните насколько полезным является альтернативный текст если изображения не видны.

Вы можете использовать атрибуты width и height, чтобы указать ширину и высоту вашего изображения . Ширину и высоту вашего изображение можете найти различными способами. Например на Mac можно использовать **Cmd** + **I**, чтобы получить информацию по изображению. Повторяя наш пример, мы можем сделать так:

```
<img src="images/dinosaur.jpg"
    alt="The head and torso of a dinosaur skeleton;
        it has a large head with long sharp teeth"
    width="400"
    height="341">
```

Это не приводит к большой разнице в отображении при нормальных обстоятельствах. Но если изображение не будет показано, например, когда пользователь только что перешёл на страницу, а оно ещё не успело загрузится, вы укажите браузеру оставить место для отрисовки изображения:



Это хорошая практика, в результате страница загрузится быстрей и более гладко.

Однако, вы не должны изменять размеры ваших изображений используя HTML-аттрибуты. Если вы установите размер изображения слишком большим, то в конечном итоге вы столкнётесь с изображениями, которые выглядят зернистыми, размытыми, или слишком маленькими, и потратите трафик для загрузки изображения, которое не будет соответствовать нуждам пользователя. Конечное изображение может также выглядеть искажённым, если вы не сохраните правильное соотношение сторон. Рекомендуется использовать графический редактор для подгонки изображения к нужному размеру, перед вставкой его на вашу вэб-страницу.

Можно также добавить атрибут title для изображений, чтобы обеспечить дополнительную подтверждающую информацию если это необходимо. В нашем примере, мы можем это сделать так:

```
<img src="images/dinosaur.jpg"
    alt="The head and torso of a dinosaur skeleton;
        it has a large head with long sharp teeth"
    width="400"
    height="341"
    title="A T-Rex on display in the Manchester University Museum">
```

Это даст нам подсказку, также как и в ссылках:



Нет необходимости писать заголовки изображений. Зачастую лучше включить такого рода вспомогательную информацию в основной текст статьи, чем прикреплять её к изображению. Однако, они полезны в некоторых обстоятельствах; например, в галереях изображений когда у вас нет места для их заголовков.

Начиная разговор о заголовках, есть множество путей как вы можете добавить заголовок к своему изображению.

Лучшим решением будет использование HTML5 <figure> и <figcaption> элементов. Они были созданы исключительно для этой цели: предоставление семантического контейнера для иллюстраций, и прозрачного связывания иллюстрации с её заголовком. Наш пример выше, мог быть переписан как то так:

```
<figure>
  <img src="images/dinosaur.jpg"
       alt="The head and torso of a dinosaur skeleton;
            it has a large head with long sharp teeth"
       width="400"
       height="341">

  <figcaption>A T-Rex on display in the Manchester University
Museum.</figcaption>
</figure>
```

<figcaption> bэлемент говорит браузерам, и вспомогательной технологии, что заголовок описывает содержимое <figure> элемента.

С точки зрения доступности, заголовки и alt имеют различные предназначения. Заголовки помогают даже тем,bкто имеет возможность просматривать изображение, тогда как alt предусматривает замену функционала отсутствующего изображения. Таким образом, заголовки и alt не подразумевают под собой одни и те же вещи, потому что оба используются браузером при отсутствии изображения. Попробуйте отключить изображения в своём браузере, чтобы увидеть как это выглядит.

Тег <figure> не является изображением. Он представляет собой независимый структурный элемент, который:

- Передает смысл компактным, интуитивно понятным способом.
- Может использоваться в различных местах страницы.
- Предоставляет ценную информацию, поддерживающую основной текст.

Тег <figure> может быть несколькими изображениями, куском кода, аудио, видео, уравнением, таблицей, либо чем-то другим.

Попробуйте придумать более подходящий альтернативный текст для вашего изображения.